Data_Visualization

Fatma Ozdal

2026-03-02

I got these codes from the teacher's example sheet. Step1) Getting the raw data. First things first, I had to grab the actual numbers from Eurostat.

```
rm(list = ls())
```

install.packages("eurostat") install.packages("readr") install.packages("ggrepel") install.packages("tmap") install.packages("sf")

```
library(eurostat)
library(readr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggrepel)
```

```
## Loading required package: ggplot2
```

I used this command to pull specific dataset, one for renewable energy and one for GDP.

```
renewable_energy_shared_downloaded <- get_eurostat("sdg_07_40")
```

```
## Dataset query already saved in cache_list.json...

## Reading cache file /var/folders/mc/3b0tdm7s5299cwpmmr0nj2bh0000gn/T//Rtmpco1GYh/eurostat/7495ac1f729

## Table  sdg_07_40  read from cache file:  /var/folders/mc/3b0tdm7s5299cwpmmr0nj2bh0000gn/T//Rtmpco1GY
```

I saved them as .csv files locally. This is a lifesaver bcs if the Eurostat website updates or goes downn, my project won't break.

#getwd() #setwd("/Users/fatmaozdal/Desktop")

```r
write_csv(renewable_energy_shared_downloaded,
file = "RenewableEnergy.csv")
```

```r
renewable_energy_shared_downloaded <- read_csv(file = "RenewableEnergy.csv")
```

```
## Rows: 3108 Columns: 6
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (4): freq, nrg_bal, unit, geo
## dbl  (1): values
## date (1): TIME_PERIOD
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
names(renewable_energy_shared_downloaded)
```

```
## [1] "freq"        "nrg_bal"     "unit"        "geo"         "TIME_PERIOD"
## [6] "values"
```

```r
head(renewable_energy_shared_downloaded)
```

```
## # A tibble: 6 x 6
##   freq  nrg_bal unit  geo   TIME_PERIOD values
##   <chr> <chr>   <chr> <chr> <date>       <dbl>
## 1 A     REN     PC    AL    2004-01-01    29.6
## 2 A     REN     PC    AL    2005-01-01    31.4
## 3 A     REN     PC    AL    2006-01-01    32.1
## 4 A     REN     PC    AL    2007-01-01    32.7
## 5 A     REN     PC    AL    2008-01-01    32.4
## 6 A     REN     PC    AL    2009-01-01    31.4
```

```r
gdp_per_capita_downloaded = get_eurostat("tec00114")
```

```
## Dataset query already saved in cache_list.json...
```

```
## Reading cache file /var/folders/mc/3b0tdm7s5299cwpmmr0nj2bh0000gn/T//Rtmpco1GYh/eurostat/754cb6d300e
```

```
## Table  tec00114  read from cache file:  /var/folders/mc/3b0tdm7s5299cwpmmr0nj2bh0000gn/T//Rtmpco1GYh/
```

```r
write_csv(gdp_per_capita_downloaded,
file = "GDP.csv")

gdp_per_capita_downloaded <- read_csv(file = "GDP.csv")
```

```
## Rows: 492 Columns: 6
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (4): freq, indic_ppp, ppp_cat18, geo
## dbl  (1): values
## date (1): TIME_PERIOD
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
names(gdp_per_capita_downloaded)
```

```
## [1] "freq"        "indic_ppp"   "ppp_cat18"   "geo"         "TIME_PERIOD"
## [6] "values"
```

```r
head(gdp_per_capita_downloaded)
```

```
## # A tibble: 6 x 6
##   freq  indic_ppp            ppp_cat18 geo   TIME_PERIOD values
##   <chr> <chr>                <chr>     <chr> <date>       <dbl>
## 1 A     VI_PPS_EU27_2020_HAB GDP       AL    2014-01-01      30
## 2 A     VI_PPS_EU27_2020_HAB GDP       AL    2015-01-01      30
## 3 A     VI_PPS_EU27_2020_HAB GDP       AL    2016-01-01      30
## 4 A     VI_PPS_EU27_2020_HAB GDP       AL    2017-01-01      30
## 5 A     VI_PPS_EU27_2020_HAB GDP       AL    2018-01-01      30
## 6 A     VI_PPS_EU27_2020_HAB GDP       AL    2019-01-01      30
```

```r
View(gdp_per_capita_downloaded)
```

```r
View(renewable_energy_shared_downloaded)
```

Which Geopolitical Entities are available?

```r
unique(renewable_energy_shared_downloaded$geo)
```

```
##  [1] "AL"        "AT"        "BA"        "BE"        "BG"        "CY"
##  [7] "CZ"        "DE"        "DK"        "EA20"      "EA21"      "EE"
## [13] "EL"        "ES"        "EU27_2020" "FI"        "FR"        "GE"
## [19] "HR"        "HU"        "IE"        "IS"        "IT"        "LT"
## [25] "LU"        "LV"        "MD"        "ME"        "MK"        "MT"
## [31] "NL"        "NO"        "PL"        "PT"        "RO"        "RS"
## [37] "SE"        "SI"        "SK"        "XK"
```

```r
unique(gdp_per_capita_downloaded$geo)
```

```
##  [1] "AL"        "AT"        "BA"        "BE"        "BG"        "CH"
##  [7] "CY"        "CZ"        "DE"        "DK"        "EA19"      "EA20"
## [13] "EA21"      "EE"        "EL"        "ES"        "EU27_2020" "FI"
## [19] "FR"        "HR"        "HU"        "IE"        "IS"        "IT"
## [25] "JP"        "LT"        "LU"        "LV"        "ME"        "MK"
## [31] "MT"        "NL"        "NO"        "PL"        "PT"        "RO"
## [37] "RS"        "SE"        "SI"        "SK"        "TR"        "UK"
## [43] "US"
```

```r
tail(renewable_energy_shared_downloaded)
```

```
## # A tibble: 6 x 6
##   freq  nrg_bal unit  geo   TIME_PERIOD values
##   <chr> <chr>   <chr> <chr> <date>       <dbl>
## 1 A     REN_TRA PC    XK    2019-01-01       0
## 2 A     REN_TRA PC    XK    2020-01-01       0
## 3 A     REN_TRA PC    XK    2021-01-01       0
## 4 A     REN_TRA PC    XK    2022-01-01       0
## 5 A     REN_TRA PC    XK    2023-01-01       0
## 6 A     REN_TRA PC    XK    2024-01-01       0
```

```r
tail(gdp_per_capita_downloaded)
```

```
## # A tibble: 6 x 6
##   freq  indic_ppp            ppp_cat18 geo   TIME_PERIOD values
##   <chr> <chr>                <chr>     <chr> <date>       <dbl>
## 1 A     VI_PPS_EU27_2020_HAB GDP       US    2019-01-01     134
## 2 A     VI_PPS_EU27_2020_HAB GDP       US    2020-01-01     135
## 3 A     VI_PPS_EU27_2020_HAB GDP       US    2021-01-01     137
## 4 A     VI_PPS_EU27_2020_HAB GDP       US    2022-01-01     133
## 5 A     VI_PPS_EU27_2020_HAB GDP       US    2023-01-01     134
## 6 A     VI_PPS_EU27_2020_HAB GDP       US    2024-01-01     135
```

Step 2)

Cleaning / Filter part:

The data I got was pretty messy and full of unnecessary things we didn't need.

We are extracting the totals from our dataset by using !(means except these..).(EA20, EA21, these are the standard region code in Eurostat, big group averages(general averages of EU). I only wanted to look at individual countries(like only Germany, Holland..).

We are selecting only the total renewable energy share(pay) (REN) , (excluding subsectors like REN_TRA) We are filtering percentages units only. (PC)

This code means; do not give me the EU averages, just show me the renewable energy percentages for each country.

```r
library(dplyr)
renewable_Filtered <-
  renewable_energy_shared_downloaded %>%
  filter(
    !(geo %in% c("EA20", "EA21", "EU27_2020")),
    nrg_bal == "REN",
    unit == "PC"
  )
```

GDP(gross domestic product- per capita income) We are extracting the totals and outside European countries(like US, Japan..) I want to keep only EU countries in my list. We chose this specific indicator because it's the 'Standardized GDP per capita'. It treats the EU average as 100.

I filtered out different currencies to put all countries on the same scale. This standard Eurostat measure accounts for living costs and compares a country's wealth to the EU average.

VI (Volume Index): wealth kisiPPS (Purchasing Power Standard) EU27_2020: this is the avg of current 27 EU countries HAB (Habitant)

means that; don't give me the EU averages, the US, or Japan. Just show me the GDP per capita for individual European countries, adjusted for purchasing power."

```
gdp_Filtered <-
  gdp_per_capita_downloaded %>%
  filter(
    !(geo %in% c("EU27_2020", "EA19", "EA20", "US", "JP")),
    indic_ppp == "VI_PPS_EU27_2020_HAB"
  )
```

Step 3)

Rename columns Note that both datasets have a column named "values". This may get confusing. Both files used the word 'values' for their numbers, which is super confusing. I renamed them to renewableShare and GDP_per_capita so i could tell them apart.

(It gave an error because i run the code twice (it doesnt see "values" column anymore because it has changed))

```
renewable_Filtered <- rename(renewable_Filtered,
                             RenewableShare = values)

gdp_Filtered <- rename(gdp_Filtered,
                       GDP_per_Capita = values)
```

i checked them , they changed.

```
head(renewable_Filtered)
```

```
## # A tibble: 6 x 6
##   freq  nrg_bal unit  geo   TIME_PERIOD RenewableShare
##   <chr> <chr>   <chr> <chr> <date>               <dbl>
## 1 A     REN     PC    AL    2004-01-01            29.6
## 2 A     REN     PC    AL    2005-01-01            31.4
## 3 A     REN     PC    AL    2006-01-01            32.1
## 4 A     REN     PC    AL    2007-01-01            32.7
## 5 A     REN     PC    AL    2008-01-01            32.4
## 6 A     REN     PC    AL    2009-01-01            31.4
```

```
head(gdp_Filtered)
```

```
## # A tibble: 6 x 6
##   freq  indic_ppp            ppp_cat18 geo   TIME_PERIOD GDP_per_Capita
##   <chr> <chr>                <chr>     <chr> <date>               <dbl>
## 1 A     VI_PPS_EU27_2020_HAB GDP       AL    2014-01-01              30
## 2 A     VI_PPS_EU27_2020_HAB GDP       AL    2015-01-01              30
## 3 A     VI_PPS_EU27_2020_HAB GDP       AL    2016-01-01              30
## 4 A     VI_PPS_EU27_2020_HAB GDP       AL    2017-01-01              30
## 5 A     VI_PPS_EU27_2020_HAB GDP       AL    2018-01-01              30
## 6 A     VI_PPS_EU27_2020_HAB GDP       AL    2019-01-01              30
```

Step 4) Merging it All Together Since I had two separate files, I had to combine them into one big table.

I used an inner_join because Renewable data goes back to 2004, but the GDP data only starts in 2013. The join automatically put out the old years where we didn't have both sets of info.

inner_join() the datasets to keep only rows that are available in both datasets: merge over both geopolitical entities and time periods

why is so important? In our case, this join allows us to see the actual relationship between GDP and Renewable Energy share. Since the Renewable data starts in 2004 and the GDP data starts in 2013, using an inner_join automatically keeps only the overlapping years. This means R cleaned up everything before 2013 for us. If we hadn't renamed the columns, R would have seen two identical "values" columns and gotten confused. Now, by giving them unique names, everything is clear and well-organized.

```
eu_combinedData <- inner_join(renewable_Filtered,
                              gdp_Filtered,
                              by = c("geo", "TIME_PERIOD"))

head(eu_combinedData)
```

```
## # A tibble: 6 x 10
##   freq.x nrg_bal unit  geo   TIME_PERIOD RenewableShare freq.y indic_ppp
##   <chr>  <chr>   <chr> <chr> <date>               <dbl> <chr>  <chr>
## 1 A      REN     PC    AL    2014-01-01            31.9 A      VI_PPS_EU27_2020~
## 2 A      REN     PC    AL    2015-01-01            34.9 A      VI_PPS_EU27_2020~
## 3 A      REN     PC    AL    2016-01-01            37.0 A      VI_PPS_EU27_2020~
## 4 A      REN     PC    AL    2017-01-01            35.8 A      VI_PPS_EU27_2020~
## 5 A      REN     PC    AL    2018-01-01            36.6 A      VI_PPS_EU27_2020~
## 6 A      REN     PC    AL    2019-01-01            38.0 A      VI_PPS_EU27_2020~
## # i 2 more variables: ppp_cat18 <chr>, GDP_per_Capita <dbl>
```

Now our dataset is ready for further analysis!

Making them readable Replace country abbreviations by full country names We should replace the country codes with full names to make the charts more user-friendly.

```
head(eu_countries)
```

```
## # A tibble: 6 x 3
##   code  name     label
##   <chr> <chr>    <chr>
## 1 BE    Belgium  Belgium
## 2 BG    Bulgaria Bulgaria
## 3 CZ    Czechia  Czechia
## 4 DK    Denmark  Denmark
## 5 DE    Germany  Germany (until 1990 former territory of the FRG)
## 6 EE    Estonia  Estonia
```

efta(european free trade association)

```
head(efta_countries)
```

```
## # A tibble: 4 x 3
##   code  name          label
```

```
##   <chr> <chr>         <chr>
## 1 IS    Iceland       Iceland
## 2 LI    Liechtenstein Liechtenstein
## 3 NO    Norway        Norway
## 4 CH    Switzerland   Switzerland
```

merge EU countries with

European Free Trade c("UK",add UK manually (we have no UK so we dont need add it)

rbind(row bind)

I replaced country codes (like AT, BE) with full country names (Austria, Belgium) to make the visualizations easier to read and more professional.

```
eu_combinedData_withnames <- left_join(eu_combinedData,rbind(eu_countries, efta_countries),
join_by(geo== code))
unique(eu_combinedData_withnames$name)
```

```
##  [1] NA            "Austria"     "Belgium"     "Bulgaria"    "Cyprus"
##  [6] "Czechia"     "Germany"     "Denmark"     "Estonia"     "Greece"
## [11] "Spain"       "Finland"     "France"      "Croatia"     "Hungary"
## [16] "Ireland"     "Iceland"     "Italy"       "Lithuania"   "Luxembourg"
## [21] "Latvia"      "Malta"       "Netherlands" "Norway"      "Poland"
## [26] "Portugal"    "Romania"     "Sweden"      "Slovenia"    "Slovakia"
```

For reproducibilty, always save your final dataset so you can use it later, even if there were changes in the data on the Eurostat website!!!

write_csv belongs to the readr package!

```
write_csv(eu_combinedData_withnames,
file = "eu_energyData.csv")
```

Step 5)

Visualization Time!

```
unique(eu_combinedData_withnames$TIME_PERIOD)
```

```
##  [1] "2014-01-01" "2015-01-01" "2016-01-01" "2017-01-01" "2018-01-01"
##  [6] "2019-01-01" "2020-01-01" "2021-01-01" "2022-01-01" "2023-01-01"
## [11] "2024-01-01"
```

. . . ..

who are these NAs? these are all Balkan countries. Eurostat collects data from them because they are in Europe (that's why they are in the main dataset i downloaded). However, they are not in the European Union (EU) yet, and they aren't in the EFTA group either.

When i made my name dictionary, i only combined the EU and EFTA country lists. So, the names of these Balkan countries were naturally missing from it. Since R couldn't find them in the dictionary, it just labeled them as NA.

```
eu_combinedData_withnames%>% filter(is.na(name))%>% distinct(geo)
```

```
## # A tibble: 5 x 1
##   geo
##   <chr>
## 1 AL
## 2 BA
## 3 ME
## 4 MK
## 5 RS
```

Fixed the Dictionary We combined the EU and EFTA lists to create a clean list of country names. We also removed any duplicates.

Merged the Data We used left_join to match the country codes in our main data with the clean names in our dictionary.

Cleaned the Missing Parts we removed the NA values because the Balkan countries didn't have names in our list. We did this to make sure the chart only shows the specific group of countries we are studying. It keeps the visual clean and focused. So we used filter(!is.na(name.y)) to remove those Balkan countries (and any others) that didn't have a matching name in our list. (name.y: This is the new, clean name column that i just brought in from my country_names_clean dictionary using left_join.)

Created the Chart We made a bar chart for the year 2023. We used reorder to sort the countries from the highest to the lowest renewable energy share so the graph is easy to read.

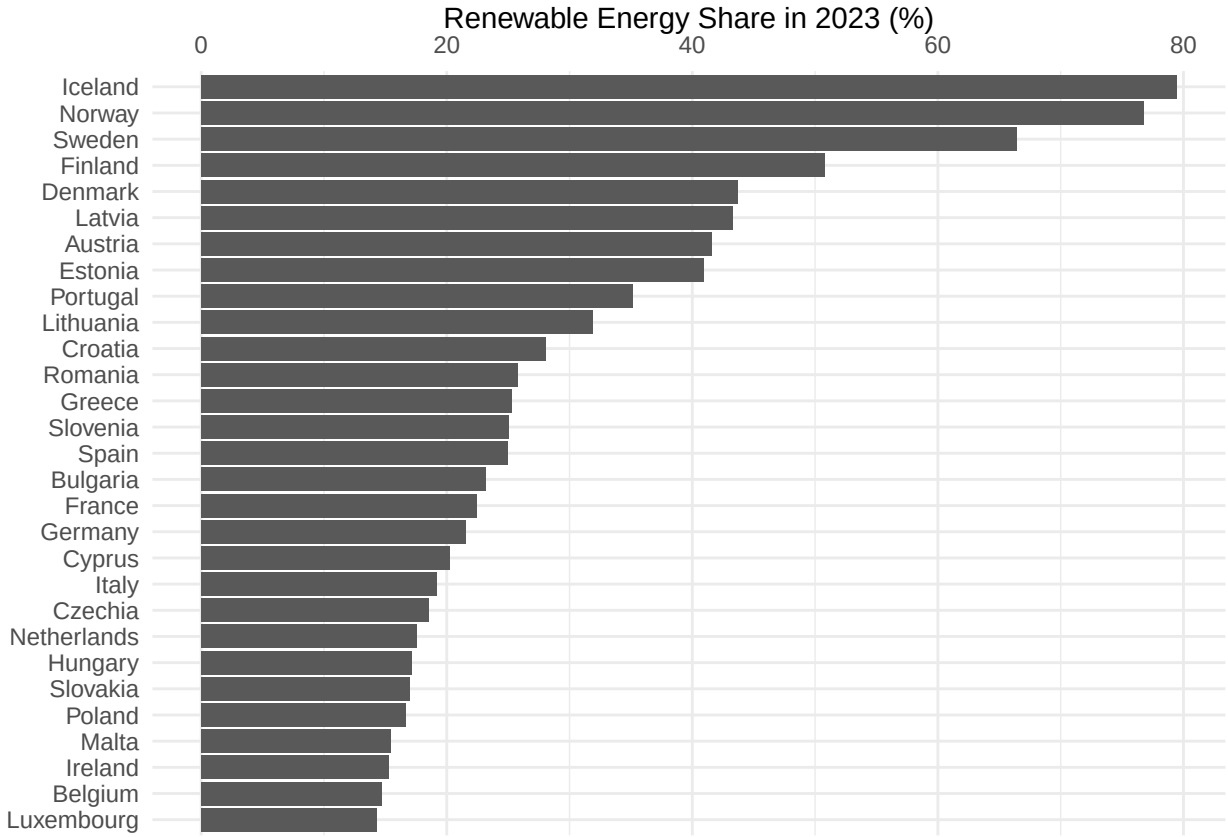
{r, eval = FALSE}; When you write eval = FALSE, you are telling the computer: "Show this code in my report, but DO NOT run it.

```
country_names_clean <- rbind(eu_countries, efta_countries) %>%
  distinct(code, .keep_all = TRUE)

eu_final_name <- left_join(eu_combinedData_withnames,
                           country_names_clean,
                           by = c("geo" = "code"))

eu_final_name <- eu_final_name %>%
  filter(!is.na(name.y))

ggplot(data = eu_final_name %>%
         filter(TIME_PERIOD == "2023-01-01"),
       mapping = aes(x = RenewableShare,
                     y = reorder(name.y, RenewableShare, FUN = "identity"))
) +
  geom_col() +
  scale_x_continuous(name = "Renewable Energy Share in 2023 (%)",
                     position = "top") +
  theme_minimal() +
  labs(y = NULL)
```



The leaders I wanted to see who's actually winning the green energy race right now. At the very top, we see Iceland and Norway. They are the clear winners, with nearly 80% or more of their energy coming from green sources. Sweden and Finland are also doing great, staying well above 50%.

surprisingly, some of the wealthiest countries are at the bottom. Luxembourg, Belgium, and Ireland have the smallest shares—all under 20%. This shows that having a lot of money doesn't automatically make a country green

There is a huge gap between the top and the bottom. Iceland is at nearly 80%, while Luxembourg is below 20%. This shows that Europe still has a long way to go to reach a balance.

The chart clearly shows which countries are leading in green energy. By using the reorder function, we can easily see the ranking from top to bottom.

The countries at the top of the chart have the highest share of renewable energy. These are the "greenest" countries in our dataset for 2023.

we can easily see the big difference between the leaders and the countries at the bottom. Some countries are clearly far ahead in their transition to renewables.

Because we filtered out the NA values (the Balkan countries we discussed), the chart looks professional and only shows the specific EU and EFTA countries we wanted to focus on.

---

names(eu_final_name)

name.x: This is the old name column coming from my eu_combinedData_withnames table.

name.y: This is the new, clean name column that i just brought in from my country_names_clean dictionary using left_join.

```
colnames(eu_combinedData_withnames)
```

```
##  [1] "freq.x"         "nrg_bal"        "unit"           "geo"
##  [5] "TIME_PERIOD"    "RenewableShare" "freq.y"         "indic_ppp"
##  [9] "ppp_cat18"      "GDP_per_Capita" "name"           "label"
```

#colnames(eu_final_name)

It was writen label=geo, but it didn't show the ful country name. I've changed it "name.y".

Adv. of Scatter Plot The Scatter plot is much better for our specific question. It answers the why: It directly shows if there is a correlation between money and energy. It spots the Outliers: We can easily point to a dot that is High Wealth but Low Green (like Luxembourg) and explain it. It saves space: One chart explains everything instead of having two separate charts.

Each dot on the chart is a country. The horizontal position (x-axis) shows how rich the country is, and the vertical position (y-axis) shows how green it is.

using geom_text is perfect because it lets the reader see exactly which dot belongs to which country.

```
library(ggplot2)

ggplot(data = eu_final_name %>% filter(TIME_PERIOD == "2023-01-01"),
       mapping = aes(x = GDP_per_Capita, y = RenewableShare)) +
  geom_point(color = "darkgreen", size = 3) +
  geom_text_repel(aes(label = name.y), vjust = -1, size = 3) +
  theme_minimal() +
  labs(title = "Relationship Between Wealth and Green Energy (2023)",
       x = "Wealth (GDP per Capita in PPS)",
       y = "Green Energy Share (%)")+
geom_smooth(method =
"lm",
formula =
"y ~ x"
,
se = FALSE,
color ="blue"
,
linetype = 2)
```

It shows the only 2023.

There is positive correlation but not too strong.

Are the richest countries actually the greenest? Do countries with more money use more renewable energy? No! (because of Luxembourg and Ireland). Actually we think that they are outliers. ????

Which countries are doing a great job with less money? It points to countries like Latvia and Estonia, who are the heroes of our story.

What is the current state of the green transition across Europe?

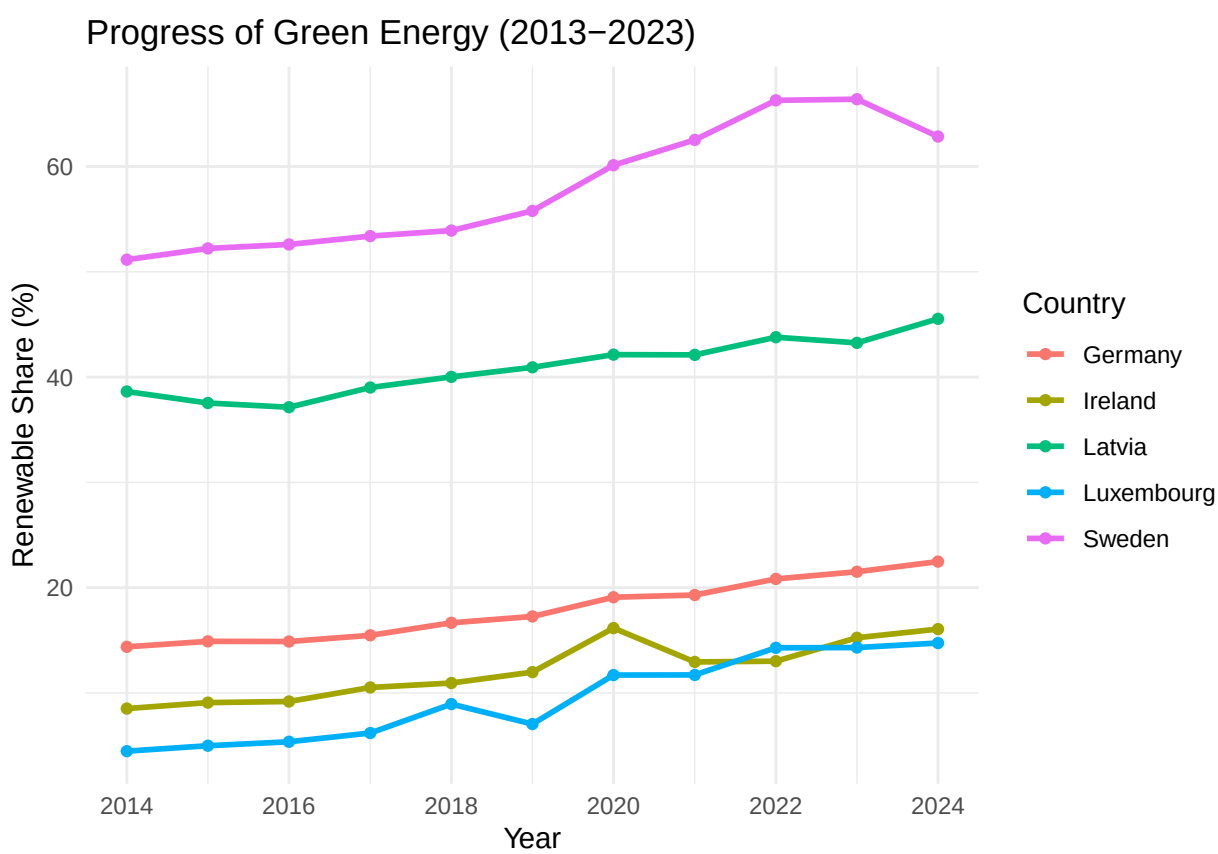
Does the economy grow at the same speed as the green transition? Since our chart is only for 2023, we are looking at a snapshot. We can say; in 2023, we don't see a parallel trend because the dots are scattered everywhere.

INTERACTIVE GROUP 2

```
target_countries <- c("Sweden", "Latvia", "Luxembourg", "Ireland", "Germany")

ggplot(data = eu_final_name %>% filter(name.y %in% target_countries),
       mapping = aes(x = TIME_PERIOD, y = RenewableShare, color = name.y, group = name.y)) +
  geom_line(size = 1) +
  geom_point() +
  theme_minimal() +
  labs(title = "Progress of Green Energy (2013-2023)",
       x = "Year", y = "Renewable Share (%)", color = "Country")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

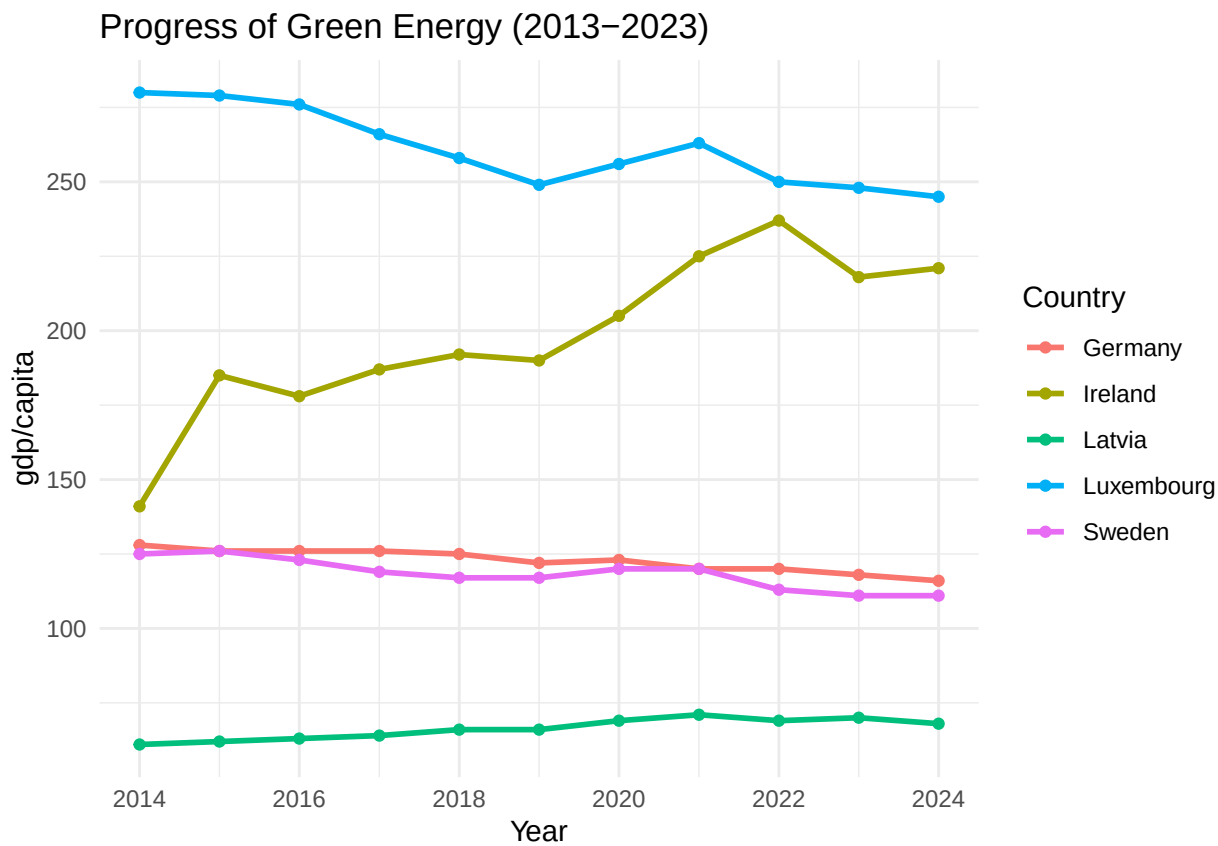


We looked at the 'Green Gap' because we wanted to see if rich countries are using their money to catch up.

We made this chart to answer one question: Is the 'Green Gap' closing? The answer is no. By comparing a less wealthy country like Latvia with the richest ones like Luxembourg, we see a surprising result: The rich countries are still stuck at the bottom. This proves my main point: Being green is a choice and a geography factor, not just a result of being wealthy. Money is a tool, but it doesn't guarantee a fast transition.

```r
target_countries <- c("Sweden", "Latvia", "Luxembourg", "Ireland", "Germany")

ggplot(data = eu_final_name %>% filter(name.y %in% target_countries),
       mapping = aes(x = TIME_PERIOD, y = GDP_per_Capita , color = name.y, group = name.y)) +
  geom_line(size = 1) +
  geom_point() +
  theme_minimal() +
  labs(title = "Progress of Green Energy (2013-2023)",
       x = "Year", y = "gdp/capita", color = "Country")
```




```r
library(tmap)
library(sf)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
gdp2019 =eu_final_name %>% filter(TIME_PERIOD == "2019-01-01") %>% rename(id=geo)
# NUTS 0  (Country Level) boundary
json_url = "https://raw.githubusercontent.com/eurostat/Nuts2json/master/pub/v2/2024/3035/20M/0.json"
europe_map = read_sf(json_url)
```

```
## Warning in CPL_read_ogr(dsn, layer, query, as.character(options), quiet, :
## automatically selected the first layer in a data source containing more than
## one.
```

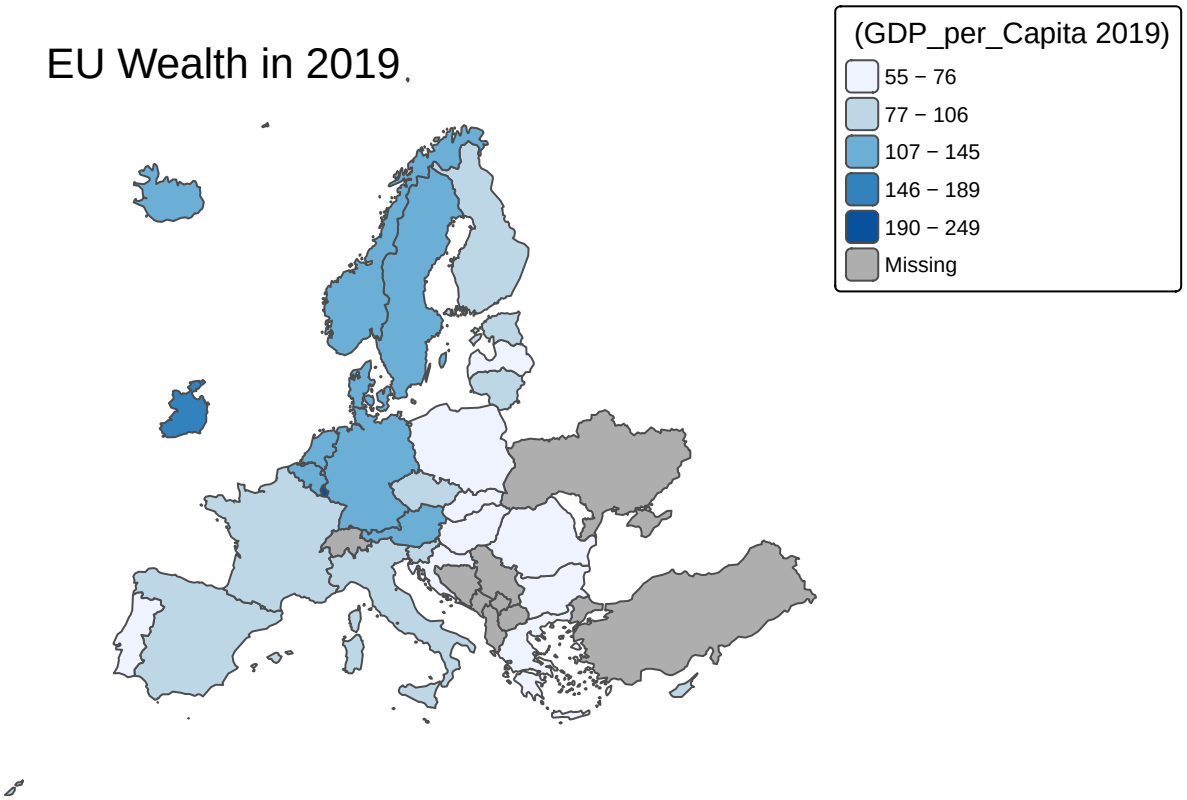
```r
# 3. combine data, make sure id match
map_data_tmap = left_join(europe_map, gdp2019, by = "id")
```

```r
tmap_mode('plot') # static map
```

```
## i tmap modes "plot" - "view"
## i toggle with `tmap::ttm()`
```

```r
tm_shape(map_data_tmap) +
  tm_polygons(
    fill = "GDP_per_Capita",
    fill.scale = tm_scale_intervals(
      style = "jenks",
      values = "brewer.blues"
    ),
    fill.legend = tm_legend(
      title = " (GDP_per_Capita 2019)"
    ),
    col = "grey30"
  ) +
  tm_layout(
    legend.outside = TRUE,
    frame = FALSE,
    title = "EU Wealth in 2019",
    inner.margins = c(0, 0, 0.1, 0) # add space on top to prevent title touching the map
  )
```

```
## [v3->v4] `tm_layout()`: use `tm_title()` instead of `tm_layout(title = )`
```


EU Wealth in 2019
(GDP_per_Capita 2019)
55 − 76
77 − 106
107 − 145
146 − 189
190 − 249
Missing